SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA0306	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Y SRAVAN KUMAR	**Reg. No.:**	192472289
Section / Batch:	CSE AI 2024	**Date:**	31-08-2026

Code of Conduct

I __Y SRAVAN KUMAR (192472289)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____SRAVAN Y_____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

Identify all entities and referring expressions.

1. Generate possible antecedents for each pronoun.
2. Apply gender, number, recency, and semantic constraints.
3. Select the most appropriate antecedent.
4. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."
Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking
Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.
Apply your algorithm to:
User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."
Your algorithm should maintain the following slots:
Departure City
Destination City
Travel Date
Number of Passengers
Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	3	3	3	3	3	15	75
2	3	3	3	3	3	15	
3	3	3	3	3	3	15	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Signature: Dr.T.Kumaragurubaran_____	Signature: _____	Signature: _____
Date: __31-08-2026_____	Date: _____	Date: _____

1. Reference Resolution Using Multiple Constraints

Pseudocode

START

Input the discourse:

"Meena gave Kavya a laptop because she needed one for her project.

She thanked her and started using it."

1. Identify all entities:

   Meena, Kavya, laptop, project

2. Identify referring expressions:

   she, one, She, her, it

3. Generate possible antecedents:

   she → Meena / Kavya

   one → laptop

   She → Meena / Kavya

   her → Meena / Kavya

   it → laptop

4. Apply constraints:

   Check Gender:

      Meena and Kavya are female → both valid for "she"

   Check Number:

      Both are singular → both valid

   Check Recency:

      Prefer the most recently mentioned suitable entity

   Check Semantic Compatibility:

      "needed one for her project" → one = laptop

      "thanked her" → her should refer to the other person

      "using it" → it = laptop

5. Select the most appropriate antecedent:

   she → Kavya

   one → laptop

   She → Kavya

   her → Meena

   it → laptop

6. Generate resolved discourse:

"Meena gave Kavya a laptop because Kavya needed one for her project.
Kavya thanked Meena and started using the laptop."

END

Step 1: Entities and Referring Expressions

Referring Expression	Possible Antecedents
she	Meena, Kavya
one	laptop
She	Meena, Kavya
her	Meena, Kavya
it	laptop

Step 2: Constraint Application

Gender: Both Meena and Kavya are female, so gender alone cannot decide **she**.

Number: Both are singular, so number also cannot decide.

Recency: The system considers the nearest suitable person mentioned before the pronoun.

Semantic compatibility: “needed one for her project” makes **one** refer to the laptop. “started using it” also strongly indicates that **it** refers to the laptop.

Discourse meaning: The natural interpretation is that **Kavya needed the laptop, thanked Meena, and then used the laptop**.

Step 3: Final Resolved Discourse

“Meena gave Kavya a laptop because Kavya needed one for her project. Kavya thanked Meena and started using the laptop.”

How the Algorithm Avoids Incorrect Resolution

When two antecedents have the same **gender and number**, the algorithm does not depend only on those features. It also checks **recency, grammatical role, semantic compatibility, and overall discourse meaning**. This combination helps select the antecedent that makes the complete discourse more logical and avoids choosing a reference only because it happens to be the nearest matching name.

2. Algorithm for Entity Chains and Discourse Coherence

Pseudocode

START

Input the discourse:
"Anjali bought a new laptop. The laptop had a powerful processor.
She installed Python on it. Later, Anjali used the computer
to develop a machine learning application."

1. Identify entities and referring expressions:
   Anjali, laptop, processor, She, it, Anjali, computer,
   Python, machine learning application

2. Link expressions referring to the same entity:
   She → Anjali
   it → laptop
   computer → laptop

3. Construct entity chains:
   Chain 1: Anjali → She → Anjali
   Chain 2: laptop → The laptop → it → computer
   Chain 3: Python
   Chain 4: processor
   Chain 5: machine learning application

4. Check entity continuity across sentences:
   Count how often important entities continue
   from one sentence to another.

5. Evaluate coherence:
   If important entities are connected across sentences,
   mark the discourse as coherent.
   Otherwise, mark it as less coherent.

6. Result:

The discourse is coherent because Anjali and the laptop are continuously referred to throughout the text.

END

1. Entities and Referring Expressions

Expression	Entity
Anjali	Anjali
the laptop	Laptop
The laptop	Laptop
She	Anjali
it	Laptop
Anjali	Anjali
the computer	Laptop

2. Entity Chains

Anjali → She → Anjali

Laptop → The laptop → it → the computer

These chains show that the main entities continue across the sentences.

3. Entity Continuity

The two main entities, **Anjali** and **laptop**, appear or are referred to in multiple sentences.

So, the continuity is **high**, because the same entities are maintained throughout the discourse.

4. Coherence Decision

The discourse is **coherent** because the sentences are logically connected and continue discussing **Anjali, her laptop, and its use for machine learning**.

5. Final Answer

The text is coherent because the entity chains "Anjali → She → Anjali" and "Laptop → The laptop → it → computer" maintain continuity across the sentences.

How Breaking an Entity Chain Reduces Coherence

If an entity chain is broken, the reader may become confused about who or what is being discussed.

For example:

“Anjali bought a new laptop. The laptop had a powerful processor. **The car** was used to develop a machine learning application.”

Here, **“The car”** does not connect with the previous entity chain. This sudden change makes the paragraph **less coherent and difficult to understand**.

3. Algorithm for Identifying Discourse Relations

Pseudocode

START

Input the discourse.

1. Divide the text into individual sentences.
2. Identify discourse markers like:
   "As a result", "After that".
3. Compare the meaning of consecutive sentences.
4. Assign a discourse relation:
   - Cause–Effect
   - Sequence
   - Problem–Solution
   - Elaboration
5. Construct the discourse structure.
6. Check whether the relations form a logical flow.
7. Display the identified relations.

END

Application

Given:

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

Sentences	Relation
Server crashed → Users could not access application	**Cause–Effect**
Users could not access → Team restarted server	**Problem–Solution**
Team restarted server → System became available	**Sequence / Cause–Effect**

Discourse Structure

Server crashed

↓

Users could not access application

↓

Technical team restarted server

↓

System became available

Conclusion

The relations make the discourse coherent because each sentence is connected logically. The **crash causes the problem, the team provides the solution, and the system becomes available afterward**.

4. Algorithm for Dialogue Act Classification

Pseudocode

START

Input the conversation.

1. Preprocess each utterance.
2. Identify important words and sentence patterns.
3. Determine the speaker's intention.
4. Assign a dialogue act.
5. Store the dialogue-act sequence.
6. Use the act to choose the next response.

END

Application

Speaker	Utterance	Dialogue Act
User	“Hello, I need help with my assignment.”	**Greeting + Request**
Agent	“Sure, what problem are you facing?”	**Question**
User	“I do not understand machine learning.”	**Inform**
Agent	“I can explain the basic concepts to you.”	**Offer**

Dialogue-Act Sequence

Greeting + Request → Question → Inform → Offer

Conclusion

Dialogue-act recognition helps the agent understand **what the user wants to do**. Based on the dialogue act, it can select a suitable next response, making the conversation more natural and relevant.

5. Algorithm for Dialogue State Tracking

Pseudocode

START

1. Initialize all slots as empty.
2. Read each user utterance.
3. Find information related to the slots.
4. Update the matching slot.
5. Check which slots are still empty.
6. Ask a question about a missing slot.
7. Continue until all required information is collected.

END

Initial Dialogue State

Slot	Value
Departure City	Empty
Destination City	Empty
Travel Date	Empty
Number of Passengers	Empty

Apply to Conversation

User: “I want to book a flight.”

No slot information is given.

Agent: “Where would you like to travel?”

User: “I want to go to Delhi.”

Update:

Slot	Value
Departure City	Empty
Destination City	**Delhi**
Travel Date	Empty
Number of Passengers	Empty

Agent: “From which city?”

User: “From Chennai.”

Update:

Slot	Value
Departure City	**Chennai**
Destination City	**Delhi**
Travel Date	Empty
Number of Passengers	Empty

Next Appropriate Question

“What is your travel date?”

After getting the travel date, the system should ask for the **number of passengers**.

Final Conclusion

Dialogue state tracking helps the conversational agent **remember information already provided by the user**. This avoids asking the same question again and keeps the conversation coherent across multiple turns.